从能省到省出闭环：AI CR 规模化实现

标准录音 4 | 时间：2026/10/22 16:23 – 2026/10/22 16:44

章节 1：AI coding 时代 Code Review 的质量挑战

AI coding 带来的问题

- 在无法编程的时代，持续交付平台是作为标准化流程的工具去接触研发流程；在 AI coding 的时代，开始探索 AI 的应用，主要是做 AI code review 相关的内容。
- 今年年初时，很多程序员对 coding agent 带有质疑，认为它没有办法替代核心编程场景，在很多场景里表现不 OK；但到了今时今日，coding agent 已经大行其道，在 coding 领域能够切切实实地帮助提升效率、解放生产力。
- 它解放了生产力，但后面的验收能力目前没有做得特别好。大量产生代码之后，很自然想到的质检手段就是 code review。

三种 AI Code Review 形态对比

实现一个 AICR 工具本身非常简单，只需把代码发给 AI 并让它查看问题，但这种方式比较廉价、粗糙，日常工作中一些小问题直调去查没有问题；AI coding 内嵌的审查（IDE 内嵌审查工具）产生审查效果无法度量，但能转移到 coding 过程中去，更早发现问题；提交代码后流水线后置扫描，触发审查形成报告后提示发现问题，但发现问题后就形成断点，后续全靠开发人工标记问题是否修复。

审查形态	实现方式	优点	缺点
直调 AI 审查	将代码发给 AI，让其查看问题	实现简单，适合日常小问题	比较廉价、粗糙
IDE 内嵌审查	AI coding 内嵌的审查工具	能转移到 coding 过程中，更早发现问题	审查效果无法度量
流水线后置扫描	提交代码后流水线触发审查并形成报告	提示发现的问题	发现问题后形成断点，后续全靠开发人工标记

核心诉求：审查必须从发现走向闭环

- 通过 AI 去发现识别代码中的潜在问题，发现问题之后给出相关的修复建议以及修复的风险、普通代码的修复线索。
- 在开发修复完代码之后，要持续去验证跟踪修复这些问题，也要跟踪有没有产生新的问题，不能因为修复前面的问题又引入新的 bug。
- **主旨目标：不但要能省，还要省出闭环。**

AICR 效果数据

- 去年年底做总结时，AICR 已经能达成 40% 的采纳率，一个季度能产生发现修复将近 1000 个问题。
- 今年通过后续一系列优化，数据保持高额增长，最终在腾讯音乐集团内部达成规模化效果，目前能够发现的问题占比大概有 40%。

章节 2：AICR 审查能力建设

审查前：从看代码到懂业务——公开上下文拓展

- 任何 AI code review 都离不开 patch 块。原生的 patch 结构比较精简，在原来场景下只需要看到变更的摘要信息，需要展开时才点击上下展开去扩充 patch 看更多内容。
- 对于 AI 来说，原生 patch 存在两个问题：一是加减混在一起，AI 有可能会产生幻觉，特别是在年初大模型能力没那么强的时候，经常出现幻觉，没有办法理解基础上下文的意图；现在的模型能力覆盖下，模型能比较好地理解这种 patch 块了。
- 具体做法：一是语法树解析，把 patch 做切片分组归类操作；二是动态扩充，根据 token 规模及时补充 patch 块头部（包含 import 第三方库的关键信息），在变更前后做语法层级的动态扩展；将 patch 重组为增量块跟减量块分离的结构，并给增量块加上行号标注。这样的结构能大幅降低大模型理解代码的幻觉，提升代码逻辑理解的精准度。

审查中：动态上下文赋予 AI 能力边界

- 即便给了很全的初始上下文，审查过程中有些场景还是无法理解，比如跨文件调用、引用到不属于变更状态的情形，无法先知地把整个代码库都给它。
- 希望大模型 AI 能够动态自主地看它需要的内容，当碰到不太确定的上下文时，自主调用工具把需要的上下文拉出来，比如执行 read 等工具，执行需求管理平台，执行语法解释工具，达到认为目前信息可以做出最终审查决策。
- 语法分析工具：帮助实现 Go 代码中全量全局变量的审查规则——Go 代码里全局变量定义之后再次修改，在动态过程中引发一系列问题。通过 SST 工具让大模型调用它分析变更相关的全局变量，再去评审那些全局变量。
- 审查中的动态上下文赋予了 AI 能力边界，它可以通过工具形式按需拉取内容，而不是靠大模型本身的知识去猜行为。

多维配置驱动业务场景上下文

- 针对不同业务，给了多维配置，驱动注入不同业务场景的上下文。定义了一些人设、仓库上下文、skill 的概念、上下工作区，在工作区里可以定义项目关联的其他仓库。
- 后台配置方便业务定制自己需要的动态配置。例如酷我音乐业务线接入时，通过后台勾选需要的垂类 agent、人设、技能，接入效果在集团内部非常不错，采纳率能达到 56%。

有上下文与无上下文的 AI 审查对比案例

- 没有上下文时，AI 看到代码除了硬编码不太合适之外，很难判断代码有没有问题。
- 有上下文时，需求要满足多端跳转，AI 读取仓库文件后发现这个模块被其他模块所用，不只是酷狗音乐，全民 K 歌也可能调用到这个模块。当全民 K 歌的 APP 打开执行到这段代码时，打开到的链接是错误不可达的路径。有了需求加仓库上下文的动态背景，AI 就能判定这是一个高危潜在问题，并且具有很高的自信度。

章节 3：放大审查效果——工具层闭环

异步异构的多轮次采样

- 代码走查是固定规则的，无论跑多少次都会在同样规则下吐出一样的审查报告；但 AI 本质是预测，即便给同样的上下文，思考过程中可能思考到不同的行为分支，第一次审查没发现问题，第二次审查可能发现另外的问题。
- 针对这个特性，做多轮次采样，根据不同的评分决策采样的轮次。

多智能体的异构分工

- 即便多轮审查 100 次，可能也不会发现领域内业务内特定的规则问题。针对这些场景，做多智能体的异构分工。
- 数字类 agent：业务有一条规则是酷狗 id 64 位，用户规模已超出 32 位定义的规则，所有业务跟 id 相关的字段必须定义为 64 位及以上。这条规则在通用大模型上不会发现，但对业务非常致命。数字类专项 agent 是较优秀的垂类 agent，占总共发现问题的 35% 占比。
- Sonar 审查：是代码走查工具，原始是非常优秀的工具，能发现很精准的代码问题，但缺失智能相关的特性。通过 AICR 赋能它，让采纳率更高，更完美地承担质量率的动作。它本质是代码走查之后的结果，提供了 87% 的采纳率。
- 部署归因 agent：跟质量关系不大，在部署失败或部署过程中发现跟代码相关的问题。部署失败后通过快速 ASR 发现问题原因是什么，推送给开发。主要效果是提升修复部署测试环境的效率，经过 AICR 报出的问题，部署修复的平均时长能小于 1 分钟。
- 辅助类 agent：闭环追踪 agent，自动追踪历史，最终在这个过程中有没有产生新的问题。
- 异构的绝对分工是为了减少重复偷看、增加效果输出，也可以独立统计和方便后续运营。

Agent 类型	核心功能	关键效果
数字类 agent	审查业务特定规则（如酷狗 id 64 位）	占总共发现问题的 35%
Sonar 审查 agent	代码走查，发现精准代码问题	提供 87% 的采纳率
部署归因 agent	部署失败时发现代码相关问题并推送开发	部署修复平均时长小于 1 分钟
闭环追踪 agent	自动追踪历史，检查是否产生新问题	—